Contextual Bandit Event Recommendation for JoinIn

Marco Carraro [1] **Francesco Vezzani** [1]

Abstract

We build a reinforcement-learning event recommender for JoinIn, a Flutter/Firebase app where students find activities through a swipe feed. The existing Firestore query stays as a hard filter, and a Contextual Thompson Sampling re-ranker decides the order of the events that pass it. Ranking uses user, event, time, popularity, and safety features that the app already stores. User actions map to shaped rewards. For this class project we leave the remote logging hooks as no-ops and evaluate the policy offline with a synthetic simulator, instead of changing the Firebase backend.

1. Goal and Setting

JoinIn shows each user a short feed of candidate activities. Before this work, the feed was sorted mainly by event time, after dropping events that were expired, full, reported, owned by the user, or already joined. Our goal is to turn this fixed order into a learning recommender: one that favors events the user is likely to enjoy while still exploring uncertain ones.

Because real user interactions cannot be collected within the class-project window, we evaluate the recommender entirely on synthetic data. A local simulator generates users, events, and contextual signals that mirror the structure of the JoinIn feed, and a ground-truth utility function not exposed to the policy assigns actions to impressions using the same reward shaping the deployed app would log. This lets us exercise the full pipeline (feature extraction, ranking, reward mapping) without touching the Firebase backend or collecting new behavioral data from real students.

2. Critical Issues

The project has a few inherent risks that influenced both the algorithm choice and the evaluation design:

- **No production reward signal**: we cannot observe real swipes, joins, leaves, or reports within the class-project window, so the agent is trained and evaluated on a hand-crafted simulator. Any conclusion is conditional on that simulator's faithfulness.
- **Sim-to-real gap**: the synthetic user model is a logistic function over the same features the policy sees. This bounds the achievable lift, removes feature-induced bias, and likely underestimates the noise a production deployment would face.
- **Sparse and shaped feedback**: most actions in the deployed app are passes and impressions, and the reward for a join only resolves after the event happens. Long-horizon and delayed signals are outside the contextual-bandit assumption.
- **Slate and position bias**: the feed shows five cards at once, so an event's outcome depends on its neighbors and on its slot. Treating each card as an independent one-step decision is a deliberate simplification.
- **Privacy and policy review**: training on real interactions requires backend changes, an opt-in analytics pipeline, and a privacy-policy update. This is outside the project scope but constrains what can actually ship.
- **Cold start and catalog churn**: the JoinIn event catalog turns over weekly, so the bandit must remain useful with very thin per-event history, which favors generalising features over per-arm estimates.

3. RL Formulation

At each step the app sees a user u and a candidate event e that passed the filter. We describe the pair with a context vector

$$x_{u,e} = [\text{profile}(u), \text{event}(e), \text{time}(e), \text{social}(e), \text{safety}(e)].$$

The action is the order of events in the feed. The reward comes from what the user does next with the card.

[1]University of Padua, Padua, Italy. Correspondence to: Marco Carraro <marco.carraro.23@studenti.unipd.it>, Francesco Vezzani <francesco.vezzani.1@studenti.unipd.it>.

Proceedings of the 38^{th} International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).

Table 1. Reward shaping used by the implementation.

Action	Reward
Impression	0.0
Join free activity	1.0
Request to join	0.7
Maybe/save	0.3
Pass/swipe left	-0.1
Leave after join	-0.6
Report activity	-1.0
Chat activity after join	0.2

Algorithm 1 Feed re-ranking with Contextual Thompson Sampling

Input: user u, filtered events E
for each event $e \in E$ **do**
  Build feature vector $x_{u,e}$
  Sample weights $\tilde{\theta}$ around the current posterior
  Compute score $s_e = x_{u,e}^T \tilde{\theta}$
end for
Sort events by s_e descending
On user action, observe shaped reward r
Update posterior mean and per-feature variance with (x_{u,e^*}, r)

4. Algorithm Choice

We use Contextual Thompson Sampling with a linear reward model (Agrawal & Goyal, 2013; Li et al., 2010):

$$\hat{r}(u, e) = x_{u,e}^T \theta.$$

For each candidate, the app draws a perturbed weight vector $\tilde{\theta}$ from a light posterior approximation and ranks by $x_{u,e}^T \tilde{\theta}$. High-scoring events go first, but uncertain yet plausible events still get a chance to appear.

We chose this method because JoinIn has a changing event catalog, sparse feedback, and strict product rules. A contextual bandit fits well: each swipe is a single one-step decision, the candidate set turns over quickly, and a first deployment must stay interpretable and safe. Compared to deep policy-gradient methods, linear Thompson Sampling needs orders of magnitude fewer interactions to commit to a useful ranking, exposes weights that the product team can inspect, and ships in tens of kilobytes through Remote Config. The generic ranking step comes from the published Dart package `dart_rl` version 1.0.0 (Vezzani, 2026); the app only adds JoinIn-specific features, priors, and logging.

5. Heuristic Baseline

The baseline reproduces the ordering JoinIn's Firestore query produced before this work: candidates that pass the hard filters are sorted strictly by ascending hours-until-event. We pick this rule on purpose rather than a uniform-random ordering because it is the rule the product actually shipped, it is what real users had been exposed to, and it already encodes a non-trivial inductive bias: events that start soon tend to be more salient and have higher attendance.

Building the baseline this way matches the project specification's "educated guess" requirement: no learning, no exposure to rewards, but informed by the domain. It is also the right benchmark from a product standpoint, since any RL agent we deploy must beat the previous shipped behaviour, not a strawman. Implementation is a single sort and lives in `baseline.py`, which runs the same synthetic stream as `evaluate.py` and writes `results/baseline_metrics.json`.

6. Implementation

6.1. Codebase Integration

We take the reusable linear Thompson Sampling code from the published `dart_rl` package and keep all product-specific logic in JoinIn:

- **Dependency**: `pubspec.yaml` adds `dart_rl: ^1.0.0`.
- **Reusable package**: `dart_rl` provides the linear Thompson Sampling sampler, bandit-arm wrapper, and ranking helper used by the service.
- **Recommendation service**: feature extraction, prior weights, reward mapping, and logging hooks for the local simulator.
- **Home feed**: runs the re-ranker after candidate retrieval and records feed interactions.

Candidate retrieval stays in `FirebaseApi.fetchEventsForFeed`. The query filters by future date, visible universities, ownership, participation, report threshold, user reports, and capacity. The re-ranker only ever sees candidates that already passed these checks.

6.2. State Features

The feature vector is small and uses only data already in `EventModel` and `UserModel`: a bias term, university match, event type, time-to-event, evening and weekend flags, attendee ratio, waiting-list ratio, image availability, freshness, profile-category match, and a report penalty.

```
static Map<String, double> buildFeatures({
  required UserModel user,
  required EventModel event,
}) {
  final attendeeCount = event.attendees?['uids']?.length ?? 0;
  final waitingCount = event.waitings?['uids']?.length ?? 0;
```

```
  final maxAttendees = event.maxAttendees?.toDouble();

  return {
    'bias': 1.0,
    'same_university': event.university == user.university ? 1.0 : 0.0,
    'free_to_join': event.type == EventType.freeToJoin.toString() ? 1.0 :
          0.0,
    'request_to_join': event.type == EventType.requestToJoin.toString() ?
          1.0 : 0.0,
    'attendee_ratio': _clamp01(attendeeCount / capacity),
    'waiting_ratio': _clamp01(waitingCount / capacity),
    'report_penalty': _clamp01((event.reportCount ?? 0) / 5.0),
  };
}
```

6.3. Ranking

The service holds an interpretable prior weight vector and one standard deviation per feature. On each feed refresh it hands the ranking step to `dart_rl`, which samples around the prior and sorts events by the sampled score.

```
static List<RankedEvent> rankEvents({
  required List<EventModel> events,
  required UserModel user,
}) {
  final bandit = LinearThompsonSampling<EventModel>(
    weights: _priorWeights,
    standardDeviations: _posteriorStd,
  );
  final arms = events.map((event) => ContextualBanditArm(
    item: event,
    features: buildFeatures(user: user, event: event),
  )).toList();
  return bandit.rank(arms).map((ranking) {
    return RankedEvent(
      event: ranking.item,
      score: ranking.score,
      exploitationScore: ranking.expectedReward,
      explorationBonus: ranking.explorationBonus,
      features: ranking.features,
    );
  }).toList();
}
```

6.4. No Server-Side Logging

Our early analysis showed that real pass and maybe signals live only in the local Isar store, and that production training would need centralized logs. For this class project we chose not to touch the Firebase backend or add new server collections. So the app-side logging methods are no-ops by default, and we evaluate with the local synthetic simulator included in the submission folder.

```
static Future<void> logInteraction({
  required UserModel? user,
  required EventModel event,
  required RecommendationAction action,
  required String sessionId,
}) async {
  debugPrint('Skipped remote logging; synthetic simulations are used.');
}
```

6.5. Training and Evaluation Scripts

The submission ships three Python entry points required by the project specification, all seeded with 0 and depending only on the standard library:

- `baseline.py` runs the chronological heuristic and writes `results/baseline_metrics.json`.
- `train.py` runs an online linear Thompson Sampling loop with SGD updates on the posterior mean and multiplicative shrinkage on the per-feature standard deviation, then saves `weights/cts_weights.json` and the windowed learning curve as `results/learning_curve.csv`.
- `evaluate.py` reloads the trained weights, replays the same seeded stream, and writes `results/evaluation_metrics.json`.

Table 2. Synthetic simulation results with fixed seed 0. Each policy is evaluated on 6,000 impressions. Delta is CTS (frozen weights) minus baseline.

Metric	Baseline	CTS	Delta
Join rate	41.53%	68.52%	+26.98 pp
Request rate	14.58%	0.03%	-14.55 pp
Pass rate	43.88%	31.45%	-12.43 pp
Avg. reward/impression	0.474	0.654	+0.180

To reproduce the reported numbers end-to-end, run the three scripts in order from the submission root (Python 3, standard library only):

```
python baseline.py
python train.py
python evaluate.py
```

7. Evaluation Plan and Results

We checked that the implementation builds: `flutter pub get` resolves the published `dart_rl` 1.0.0 package from pub.dev, and the Flutter analyzer reports only pre-existing style hints in `home_page.dart`, not type errors from the recommender.

Because we do not change the Firebase server, evaluation runs through the local Python scripts described above, all reseeded to 0 as required by the project specification. The simulator creates 1,200 users with 30 candidate events each. Each policy shows the top five events per user, giving 6,000 impressions per policy. Users have a university and a preferred category; events have category, university, type, time, popularity, freshness, image availability, and report risk. A ground-truth utility function not exposed to the policy then picks each action using the same reward shaping as the app. We measure agent quality with average reward per impression: it aggregates the full reward table of Section 2 into a single scalar and stays interpretable across windows.

8. Discussion

The learning curve in Figure 1 answers the project specification's "learning over time" requirement directly: the

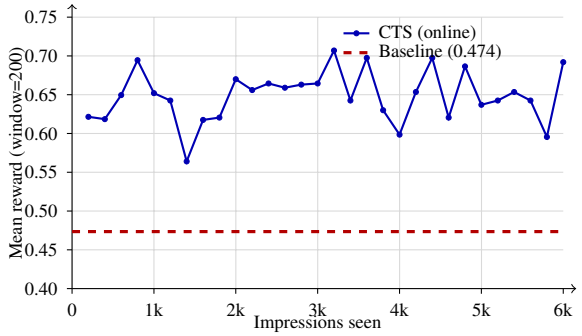

Figure 1. Online learning curve: windowed (200-impression) mean reward of CTS during training compared with the chronological baseline (horizontal line). The bandit overtakes the baseline within the first 200 impressions and stabilises around 0.65.

baseline is a flat reference at 0.474, and the bandit's windowed reward climbs above it within the first window of 200 impressions and stabilises near 0.65. The variance across windows is the residual exploration noise that the diagonal posterior never collapses entirely, which is intentional: any production version of the agent must keep some uncertainty to cope with the JoinIn catalog turning over weekly.

Table 2 shows the trained policy lifts the average reward by +0.180 per impression (+38%) and the join rate by +27 percentage points relative to the heuristic. The drop in request rate is the dominant side-effect: once the posterior commits, free-to-join events crowd out request-to-join events because the former carries a +1.0 reward versus +0.7. The bandit is therefore behaving rationally given the reward shaping, but the result also exposes a real product question — whether to re-balance the reward table, add a per-type fairness constraint, or expose request-to-join events under a separate budget. We surface this as an issue rather than a bug.

The RL agent beats the heuristic for two reasons that go beyond raw averages: it personalises (the same event is ranked differently for two users depending on `same_university` and `category_profile_match`), and it adapts (the SGD update on the posterior mean lets it correct miscalibrated priors without a redeploy). The heuristic does neither. Possible improvements include: (i) a full Bayesian linear regression with a precision-matrix posterior instead of the diagonal SGD approximation, (ii) a position-aware reward that discounts cards further down the stack, (iii) a per-arm fairness constraint to protect request-to-join exposure, and (iv) lightweight non-linear features (e.g., category $\times$ time-of-day cross-terms) before moving to a neural ranker.

9. Limitations and Next Steps

The current model uses a diagonal posterior and SGD updates rather than the full Bayesian linear regression in Algorithm 1. This is safe for a first deployment and fits a class project that avoids server changes, but it leaves correlation structure between features on the table. The synthetic simulator checks the mechanics and the reward design; it cannot prove a production lift because the user model is itself a linear function of the same features the policy sees. The Q4 release should: update the privacy policy and backend data contract; add an opt-in analytics pipeline; estimate θ from real past interactions; ship updated weights through Remote Config or a server endpoint; and model slate effects and position bias.

References

Agrawal, S. and Goyal, N. Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning*, pp. 127–135, 2013.

Li, L., Chu, W., Langford, J., and Schapire, R. E. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670, 2010.

Vezzani, F. dart_rl: Reinforcement learning algorithms for dart. `https://pub.dev/packages/dart_rl`, 2026. Version 1.0.0, published on pub.dev.